# Stack Demo
# Test Plan

---

E-Commerce Web Application Testing

https://bstackdemo.com/

## 1. Target Platform URL

https://bstackdemo.com/

## 2. System Description

Stack Demo is an e-commerce web platform used to test core functionalities such as user authentication, product catalog browsing, vendor filtering, cart management, checkout workflow, and order history tracking.

## 3. Testing Focus

Verifying core functional modules and ensuring system stability across manual, automation, API, and cross-browser test execution cycles.

## 4. Project Purpose — Core Purpose & Quality Objectives

| Item | Core Purpose & Quality Objectives |
|---|---|
| Objective 1 | Verify that all user-level shopping workflows work as expected. |
| Objective 2 | Ensure data integrity across UI, API, and cart calculation layers. |
| Objective 3 | Identify UI, functional, cross-browser, and API defects. |
| Objective 4 | Validate checkout simulation, shipping details entry, and order confirmation flows. |

## 5. Blueprint vs Test Plan Element

| Blueprint Element | Test Plan Element |
|---|---|
| Building Design | Test Objectives: Ensure all major e-commerce features (login, product filtering, add to cart, checkout) function correctly. Verify the website meets both functional and non-functional requirements. Identify, log, and track defects using Jira. Validate stability, usability, and cross-browser compatibility before release. |

## 6. Scope of Testing

| Scope Category | Detailed Functionality Coverage |
|---|---|
| In Scope | • User registration & login (Standard, Locked, Existing orders, Fav users).<br>• Product search & vendor filtering (Apple, Samsung, Google, OnePlus).<br>• Price sorting (Lowest to Highest, Highest to Lowest).<br>• Add to Cart & Wishlist / Favourites.<br>• Checkout process & shipping form validation.<br>• Order confirmation overlay & order history management.<br>• API endpoint testing (products, product detail, signin, cart).<br>• Cross-browser testing (Chrome, Firefox, Edge).<br>• Cart persistence across page refresh.<br>• Sort + filter combination behaviour. |
| Interfaces & Workflows | • UI testing through web interface across supported desktop browsers.<br>• API interface through Python requests library.<br>• Workflow: Product Catalog → Filter/Sort → Add to Cart → Checkout → Confirmation.<br>• Vendor Integration: Mock product feeds (Stack Demo internal data). |
| Out of Scope | • Real payment gateway transaction clearance with real banking institutions.<br>• Native mobile application testing (iOS / Android native binaries).<br>• High-volume performance and load stress testing.<br>• Unit testing / Database (MySQL) validation.<br>• Admin dashboard testing. |

## 7. Testing Strategy & Types

| Type | Description |
|---|---|
| Manual testing | Functional, Exploratory, and Documented End-to-End User Scenario Testing. |
| Automation testing | Automate regression test cases for login, product filtering, cart, and checkout modules using Selenium WebDriver (Python). |
| API testing | Validate REST endpoints (products, signin, cart) using Python requests. |
| Cross-browser testing | Run the full suite on Chrome, Firefox, and Edge. |
| Non-functional testing | Basic usability and cross-browser compatibility checks. High-volume load testing remains out of scope. |

## 8. Testing Strategy — Responsibilities Example

| Role | Responsibilities |
|---|---|
| Test Team | Execute test cases, report defects. |
| Technical Team | Resolve defects and deploy builds. |
| Project Manager | Monitor progress and approve releases. |

## 9. Test Assumptions & Data Approach

| Parameter | Details & Guidelines |
|---|---|
| Test Assumptions | • Test environment is stable and accessible at https://bstackdemo.com/.<br>• All required modules are functional on the demo platform.<br>• Dummy user credentials and sample product data are available.<br>• Internet connection and target browser versions are available.<br>• Test cases must be validated against actual UI behaviour before being marked as FAIL. |
| Data Approach | • Test data will be created manually and maintained in the QA repository.<br>• Functional Testing: Dummy users, test products, and fake shipping addresses.<br>• UAT Testing: Realistic user scenarios with valid and invalid inputs.<br>• No confidential or production data will be used. |

## 10. Test Environment Specifications

| Parameter | Value |
|---|---|
| URL | https://bstackdemo.com/ |
| Browser | Google Chrome, Microsoft Edge, Mozilla Firefox |
| Automation tool | Selenium WebDriver (Python) |
| Backend | RESTful Web Services / Session Storage |
| OS | Windows 10–11, macOS |

## 11. Level of Testing — Functional & UAT

| Testing Level | Features to be Tested |
|---|---|
| Functional Testing | • User authentication (standard login, locked account error checks, logout).<br>• Product catalog filtering, brand multi-select, price sort order.<br>• Adding, updating item counts, and removing items in shopping cart.<br>• Checkout process: shipping address input, submit action, modal popup.<br>• Order confirmation display and order history tracking.<br>• Cart persistence across page refresh.<br>• Sort + filter combination. |

| Testing Level | Features to be Tested |
|---|---|
| User Acceptance Testing (UAT) | • User Account: Validate user profile selection, login, logout, and order history view.<br>• Product & Cart: Validate filtering, adding items, modifying quantities, and removing items.<br>• Checkout: Validate address entry, place order action, and confirmation display.<br>• Order Management: Confirm placed order appears correctly under existing_orders_user. |

## 12. Level of Testing — Regression Testing Matrix

| Focus Area | Regression Validation Rules |
|---|---|
| Authentication | Verify user authentication and login still function correctly after system updates or code commits. |
| Cart & Favourites | Validate that adding/removing items in the cart and favourites continues to update totals accurately. |
| Price Calculations | Ensure quantity updates and price recalculations remain completely accurate. |
| Data & Layout | Ensure product images, prices, vendor tags, and stock information display consistently. |
| Cross-Browser UI | Validate data consistency between UI display and API responses across Chrome, Firefox, and Edge. |

## 13. Execution Strategy (Entry Criteria Guidelines)

| Prerequisite Phase | Entry Benchmark Conditions |
|---|---|
| Environment Readiness | Test environment (https://bstackdemo.com/) is fully accessible, responsive, and stable. |
| Test Data Preparation | Test data for users, products, quantities, and addresses are created, verified, and ready. |
| Toolchain Setup | All testing tools (Selenium, requests, Jira, GitHub) are installed, configured, and verified. |
| Build Stability | The latest build is deployed successfully, and core modules (Login, Cart, Checkout) pass smoke checks. |
| If Conditions Are Not Met | The QA team will assess the associated risks, identify mitigation actions, and recommend whether to proceed or delay testing. |

### Entry Criteria Sign-Off Checklist

| Entry Criteria | Test Team |
|---|---|
| Test environment(s) is available and accessible | Jana Tamer Hassan |
| Test data is created and validated | John Samy |
| Code build has been merged and deployed successfully | Basem Sherif |
| Development team completed initial smoke/unit checks | Basem Sherif |
| Test scripts are completed, reviewed, and approved by QA Lead | Basem Sherif |

## 14. Execution Strategy (Exit Criteria Guidelines)

| Quality Gate | Mandatory Exit Conditions |
|---|---|
| Execution Coverage | All planned test cases have been executed across functional, regression, UAT, and API cycles. |
| Pass Rate Target | At least 90% of test cases have passed successfully. |
| Defect Threshold | No Critical or High severity defects remain open in the issue tracker. All Medium and Low defects are logged in Jira. |

| Quality Gate | Mandatory Exit Conditions |
|---|---|
| Documentation & Cleanup | All test results and execution logs are stored; final reports are signed off; environment cleaned up. |
| If Criteria Are Not Met | The QA team will assess the impact and release risk, then recommend mitigation actions or follow-up testing. |

### Exit Criteria Sign-Off Matrix

| Exit Criteria Benchmark | Responsible Entity |
|---|---|
| 100% Test Scripts executed across all planned cycles | Jana Tamer Hassan |
| 90% pass rate achieved across all executed test scripts | Basem Sherif |
| Zero open Critical and High severity defects remaining | Basem Sherif |
| All remaining defects are either cancelled or documented as Change Requests for future releases | Basem Sherif |
| All expected and actual results are captured and documented with test scripts | Farah Hossam El-Din |
| All test metrics collected based on daily and weekly status reports | Basem Sherif |
| All defects logged in Jira / Defect Tracking Spreadsheet | Fawzy Mohamed |
| Test environment cleanup completed and backup created | Jana Tamer Hassan |

## 15. Validation & Defect Management Protocol

| Aspect | Details |
|---|---|
| Execution | It is expected that testers will execute all assigned scripts in each cycle described in this plan. |
| Tracking | All discovered defects will be tracked using Jira. |
| Responsibility | It is the responsibility of the tester to open the defect ticket, retest upon fix, and officially close the ticket. |
| Verification | QA Lead verifies critical fixes before closing. |

## 16. Timeline & Milestones

| Time Period | Milestone & Core Activities |
|---|---|
| Week 1 | Test Planning & Setup: Requirements analysis, Test Plan creation, environment verification, test data preparation, and Jira sprint creation. |
| Week 2–3 | Manual Testing Execution: Exploratory testing, manual test case execution (login, filtering, cart, checkout), and defect reporting in Jira. |
| Week 4 | Automation & API Execution: Selenium WebDriver framework setup, script creation, regression execution, and API collection runs. |
| Week 5 | System & UAT Support: Cross-browser testing (Chrome, Edge, Firefox), retesting bug fixes, regression suite execution, and end-to-end UAT validation. |
| Final Week (Week 6) | Test Closure: Final defect triage, test summary report generation, exit criteria sign-off, and artifact archiving. |

## 17. Defect Severity Matrix

| Severity | Impact & Stack Demo Scenarios |
|---|---|
| Critical | System crash or core functionality failure preventing complete usage.<br>Examples: Checkout modal fails to open or crashes on submit; Add to Cart button completely unclickable; Login/Sign-in completely broken; Server error (500) appears on homepage. |

| Severity | Impact & Stack Demo Scenarios |
|---|---|
| High | Major feature not working properly, but system remains partially usable.<br>Examples: Product images fail to render for specific vendor filters; Total price calculation fails to update on qty change; Order confirmation overlay displays wrong shipping address. |
| Medium | Minor functionality or layout issue that does not halt core purchase flows.<br>Examples: Vendor filter checkbox misaligned; Product description overlaps border; Sorting dropdown slight loading lag. |
| Low | Cosmetic or low-priority issues with zero business or operational impact.<br>Examples: Typo in product description; Slight color mismatch on footer copyright; Missing favicon on browser tab. |

## 18. Project & Testing Dependencies

| Dependency Type | Critical Requirement Details |
|---|---|
| Environment & Tools | Availability and stability of https://bstackdemo.com/; full access to Jira, Selenium, and Python requests. |
| Development & Builds | Stable builds and timely defect fixes from the development team before each test cycle. |
| Data & Scripts | Clean test data prepared prior to execution; functioning mock checkout simulation scripts. |
| Team Coordination | Timely defect fix confirmations; smooth coordination within the team for Git branching and PR reviews. |

## 19. Risk Management & Mitigation

| Category | Action Plan | Responsible |
|---|---|---|
| Identified Risks | Environment downtime, script locator breaking due to UI updates, delayed bug verification. | Basem Sherif |
| Mitigation Strategy | Maintain backup dynamic XPath/CSS, schedule daily team syncs, prioritize High/Critical defect triage. | Jana Tamer Hassan |
| Contingency Plan | Re-run failed automated cases manually in alternate browsers and maintain detailed DOM logs. | Fawzy Mohamed |

## 20. Test Deliverables Matrix

| # | Deliverable Artifact | Document Content & Scope |
|---|---|---|
| 1 | Test Plan Document | Test objectives, scope, strategy, in/out-of-scope definitions, testing types, schedule, team allocation. |
| 2 | Test Scenarios | High-level journeys: Registration, Login, Vendor Filtering, Cart, Checkout, Order History, API. |
| 3 | Test Cases Matrix (42) | Test Case ID, Title, Pre-conditions, Steps, Expected/Actual Results, Pass/Fail Status. |
| 4 | Test Data Set | Credentials (demouser, locked_user, existing_orders_user, fav_user), catalog params, addresses. |
| 5 | Test Execution Report | Total executed cases, pass/fail/blocked metrics, execution dates, tester names. |
| 6 | Bug / Defect Log (Jira) | Bug ID, summary, steps, expected/actual, severity/priority, status, reporter, assigned, environment. |
| 7 | Test Summary Report | Overview of testing process, defect distribution, pass rate metrics, release sign-off. |
| 8 | API Test Report | 6 API test cases with endpoint, method, expected status, actual status. |
| 9 | Cross-Browser Report | Pass/fail matrix for Chrome, Firefox, Edge. |
| 10 | Automation Scripts | stackdemo_tests.py (Selenium + requests), requirements.txt. |